SkillMarket

README | Project overview and local setup

What is SkillMarket?

SkillMarket is a Django service marketplace where sellers publish their skills and buyers propose paid engagements. It brings service discovery, offers, engagement status, and reviews into one application. Prices are recorded, but the application does not process payments.

What problem does it solve?

Hiring a service provider can involve scattered advertisements and informal conversations with no clear record of progress. SkillMarket gives buyers a searchable service catalog and gives both parties a shared record of an offer, its outcome, and feedback after completion.

Buyer and seller roles

• **Buyer:** Browse and search published services, submit a price and message, track offers in a buyer dashboard, and leave one rating and review for each completed engagement.

• **Seller:** Create and edit service listings, publish or save them as drafts, manage incoming offers, accept or reject pending offers, and mark accepted engagements complete.

Users select a Buyer or Seller role during registration. Django groups and ownership checks control access. Staff can moderate through Django admin.

Technologies used

Backend: Python 3.12, Django 5.2 LTS, Django REST Framework, and Django authentication.
Frontend: Django templates, Tailwind CSS 4, and vanilla JavaScript.
Database: PostgreSQL through psycopg; SQLite for local evaluation. python-dotenv and dj-database-url load configuration.
Serving: WhiteNoise for static assets and Gunicorn for Linux deployment. Node.js/npm is optional for rebuilding CSS.

Features implemented

• Registration, login/logout, role-specific dashboards, and administrator moderation.

• Service listings with categories, descriptions, prices, and published/draft status; keyword search, category filtering, and paginated public discovery.

• Offer workflow: Pending to Accepted or Rejected; Accepted to Completed.

• Ratings from 1 to 5 and written reviews, restricted to the buyer after completion; duplicate reviews are prevented.

• REST endpoints for listings, offers, and reviews, with session authentication, CSRF protection, validation, and shared authorization logic.

• Transactional offer/review operations and protected engagement history. Listings with offers must be archived as drafts instead of deleted.

Install the project

Install Python 3.12. Download or clone the project and open PowerShell in the SkillMarket folder containing manage.py. The commands below use the virtual environment directly, so activation is unnecessary.

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
Copy-Item .env.example .env
```

Copy the example only for a fresh setup; preserve an existing .env. On macOS/Linux, create the environment with python3.12 and use .venv/bin/python in place of the Windows executable.

Configure environment variables

Edit .env in the project root. Django loads it automatically. For local SQLite evaluation, use:

```
SKILLMARKET_DEBUG=True
SECRET_KEY=replace-with-a-long-random-secret
ALLOWED_HOSTS=localhost,127.0.0.1
CSRF_TRUSTED_ORIGINS=http://localhost:8000,http://127.0.0.1:8000
USE_SQLITE=True
```

Generate a secret with the following command and paste its output as SECRET_KEY. Keep .env and credentials private.

```
.\.venv\Scripts\python.exe -c "import secrets;
print(secrets.token_urlsafe(50))"
```

SQLite: Leave DATABASE_URL unset or empty. SQLite is available only when debug mode and USE_SQLITE are both True. An existing DATABASE_URL takes precedence over USE_SQLITE.

PostgreSQL: Create a database, set USE_SQLITE=False, and add the connection URL below with your own credentials. URL-encode special characters in the username/password.

```
DATABASE_URL=postgresql://USER:PASSWORD@HOST:5432/DB?sslmode=require
```

Deployment settings: Set SKILLMARKET_DEBUG=False, a unique random SECRET_KEY, your domain in ALLOWED_HOSTS, and HTTPS origins in CSRF_TRUSTED_ORIGINS. Host/origin lists are comma-separated without spaces. PostgreSQL is required outside debug mode. SKILLMARKET_DEBUG takes priority over DEBUG. Supabase can host PostgreSQL; the app uses Django authentication, not Supabase Auth.

Run migrations

```
.\.venv\Scripts\python.exe manage.py migrate
```

This creates the Django and marketplace tables and provisions Buyer/Seller groups and permissions. Apply the committed migrations before registering accounts. After changing models, create a migration with manage.py makemigrations marketplace, then run migrate again.

Start the Django server

```
.\.venv\Scripts\python.exe manage.py runserver
```

Open **http://127.0.0.1:8000/**. Register separate Buyer and Seller accounts to try the full workflow. Stop the server with Ctrl+C. The development server is for local use.

Optional: create an administrator account, then sign in at /admin/:

```
.\.venv\Scripts\python.exe manage.py createsuperuser
```

Optional: load fictional demonstration listings and accounts in debug mode:

```
$env:DEMO_PASSWORD='choose-a-local-password-at-least-12-characters'
.\.venv\Scripts\python.exe manage.py seed_demo
```

The seed command creates six sellers and demo.buyer using the supplied password. It does not change existing passwords.

Compiled CSS is included. Only when changing styles, install Node.js/npm and rebuild:

```
npm.cmd ci
npm.cmd run build:css
```

Execute tests

```
.\.venv\Scripts\python.exe manage.py test marketplace
.\.venv\Scripts\python.exe manage.py check
.\.venv\Scripts\python.exe manage.py makemigrations --check --dry-run
```

The test suite covers browser forms and APIs, roles, ownership, CSRF, offer transitions, input validation, and duplicate reviews. Django uses a separate test database; PostgreSQL test credentials need permission to create it. SQLite configuration is the simplest local option.

To save per-test results and a transcript, run the optional evidence command. It overwrites test-results.json and test-output.txt in submission/03_Marketplace/evidence/.

```
.\.venv\Scripts\python.exe scripts/run_tests.py
```

Known limitations

• No payment processing, shopping cart, real-time messaging, or external authentication providers.

• Dashboards load all of a user's records without pagination. Search uses substring matching; large datasets may need database search tuning.

• SQLite is intended for local evaluation and does not provide the same row-locking behavior as PostgreSQL.

• Existing test evidence covers sequential operations; concurrent load testing and a production security audit have not been performed.

• A live deployment and Supabase connection are not established by this README. Production still needs HTTPS, static collection, hosting configuration, and database backups.